Generative and Agentic AI in Practice

DS-246 Tutorial

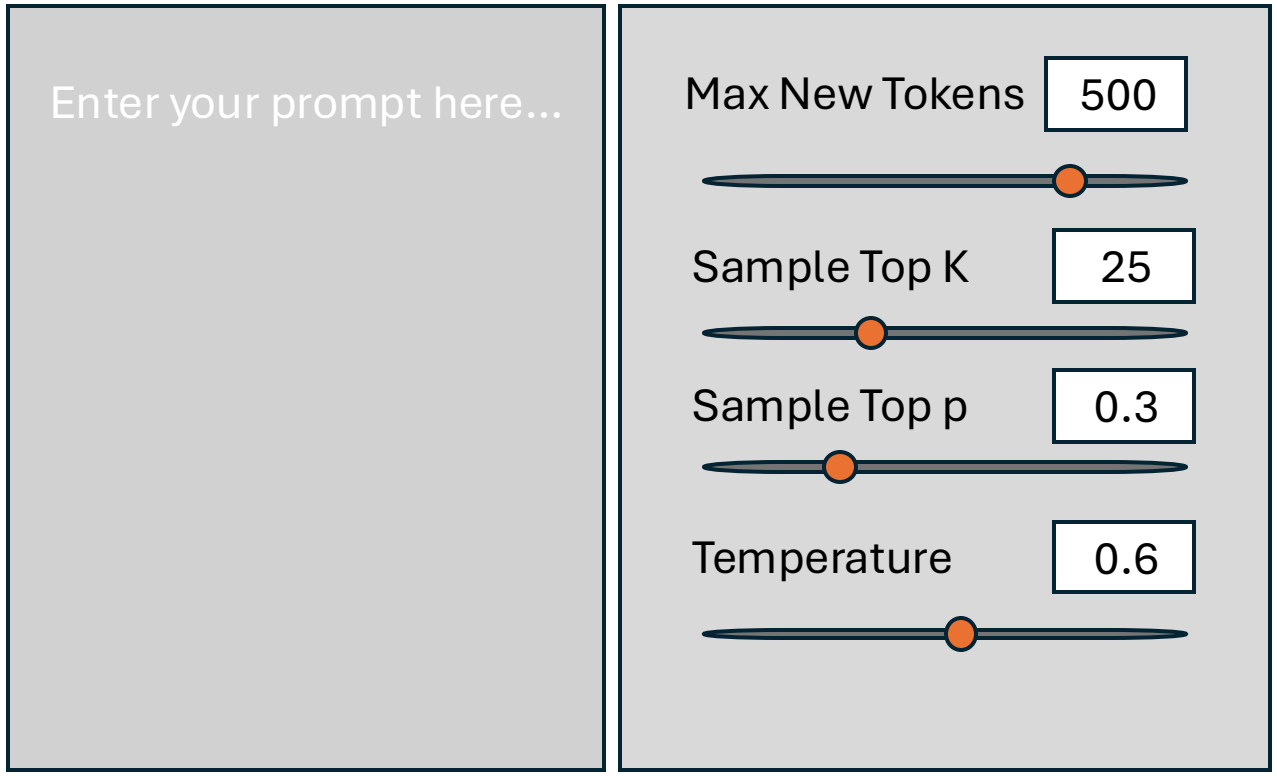
Protyush P. Chowdhury

Tutorial Contents

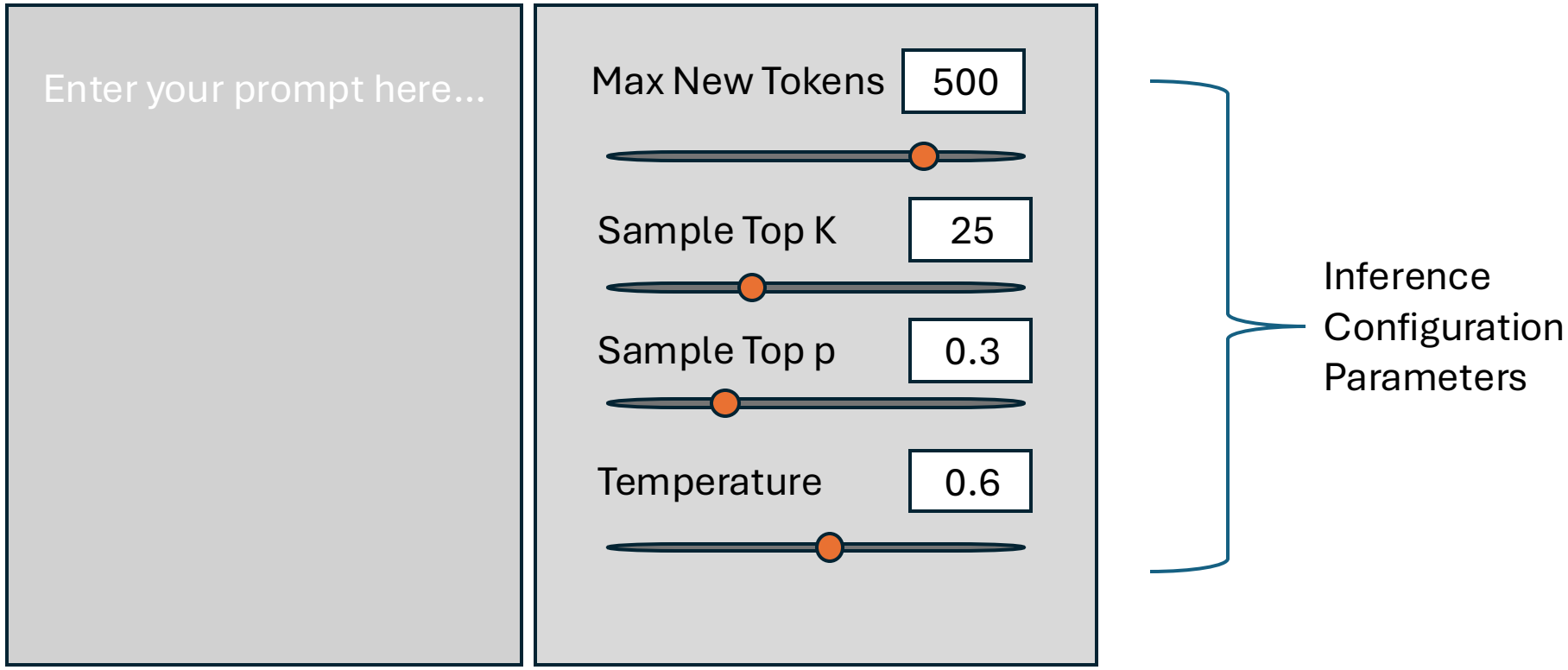
- LLM settings
  - top-p
  - Temperature
- Core Architecture of LLMs in Practice
- Scaling Laws in Model Pretraining

Generative Configuration

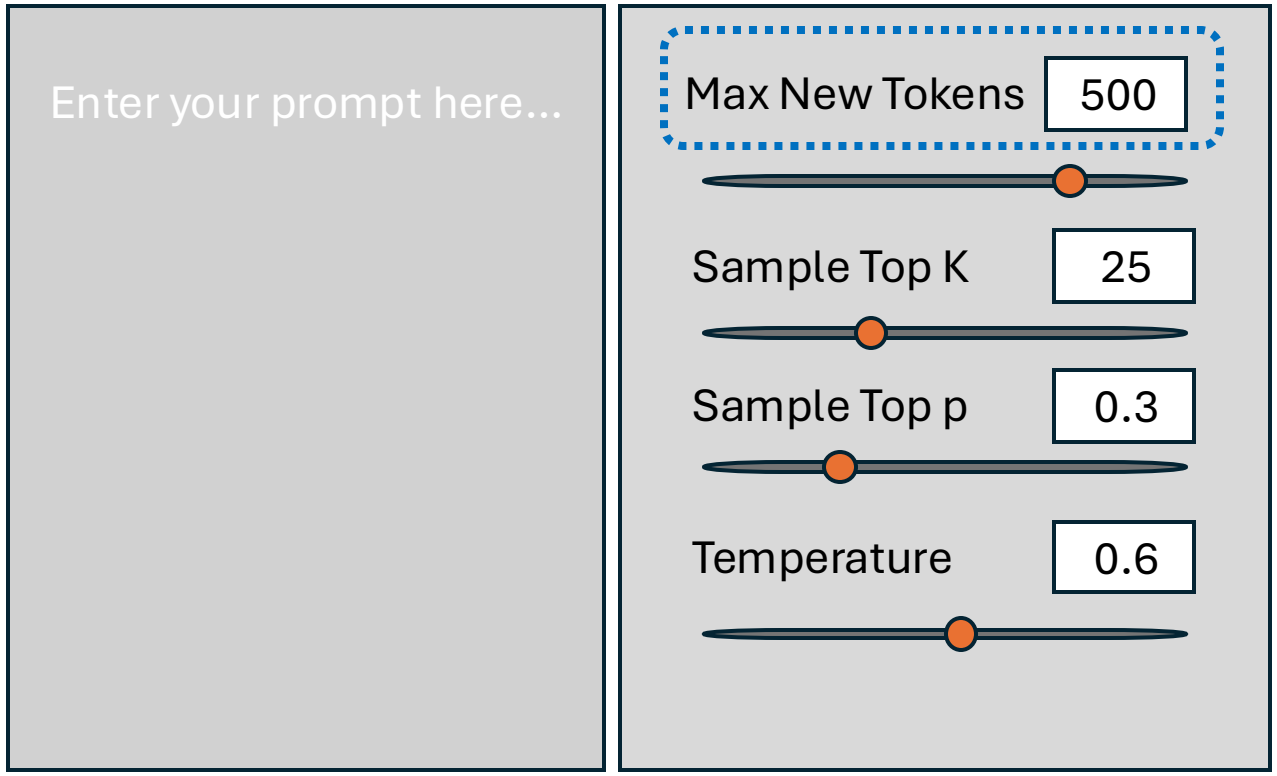
Generative Configuration



Generative Configuration



Generative Configuration



Max New Tokens

max_new_tokens = 100

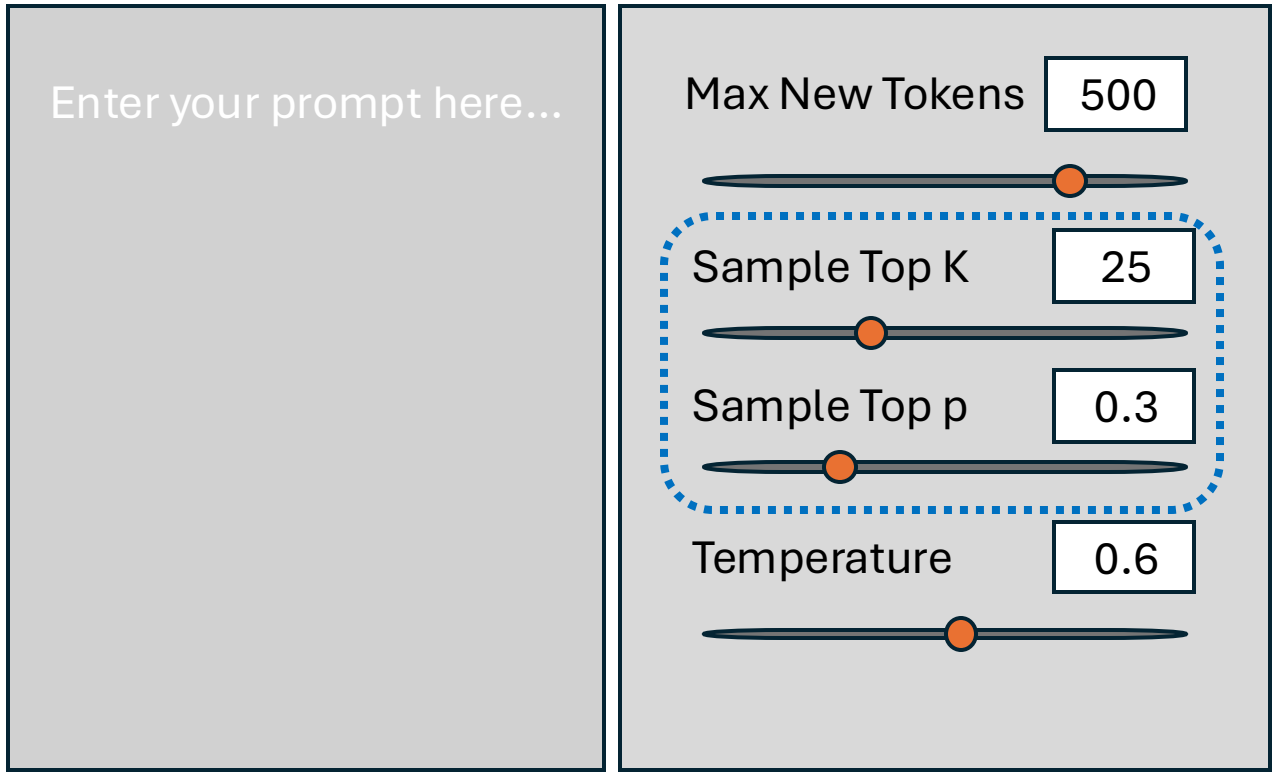
max_new_tokens = 150

max_new_tokens = 200

It's the Max number of new tokens, Not a hard number of generated tokens.

<EOS> token

Generative Configuration



Generative Configuration (Greedy vs Random Sampling)

apple	0.02
coffee	0.40
mojito	0.10
tea	0.30
……	……

Greedy Sampling: The token with highest probability is selected.

apple	0.02
coffee	0.40
mojito	0.10
tea	0.30
……	……

Random Sampling: The token is sampled according to the probability distribution (softmax output).

Here, there is a 40% chance that "coffee" will be selected.

Generative Configuration (Random Sampling Techniques)

Basic Multinomial Sampling from LLMs

Algorithm 1 Basic LLM token sampling

```
function SAMPLE_TOKENS(L)
    s ← ()
    for i ← 1, L do
        α ← LM(s, θ)
        Sample s ~ Categorical(α)
        if s = EOS then
            break
        end if
        s ← append(s, s)
    end for
    return s
end function
```

torch.multinomial

Used in :

- Standard Research
- Hugging Face Transformers library
- vLLM

Courtesy: Willard and Louf, 2023

Generative Configuration (Random Sampling Techniques)

- **Standard research / Hugging Face** → use torch.multinomial (simple & clear).
- **High-performance inference/training libraries** (Megatron-LM, FasterTransformer, DeepSpeed kernels) → sometimes use **Gumbel-Max** because it's GPU-friendly.
- **Probabilistic models (VAEs, Gumbel-Softmax papers)** → use Gumbel-Max as the foundation for categorical reparameterization.

Generative Configuration (top-k and top-p)

apple	0.02
coffee	0.40
mojito	0.10
tea	0.30
……	……

K=3

top-K: Sample space is restricted to the **K** tokens having the highest probabilities.

apple	0.02
coffee	0.40
mojito	0.10
tea	0.30
……	……

p=0.75

top-p: Sample space is restricted tokens with highest probability having a cumulative probability of **<=*p***.

Generative Configuration (Random Sampling Techniques)

In Hugging Face Transformers, calling ***model.generate(do_sample=True, ...)*** :

- They implement **top-k/top-p filtering** first.
- Then they sample using **torch.multinomial** by default.

For example, transformers.generation.logits_process → torch.multinomial is used to actually draw the next token.

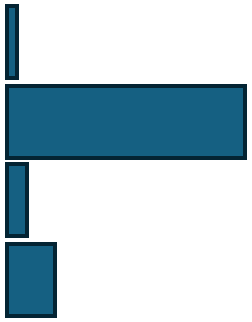
Assignment Teasers

- Code up top-p and top-K sampling techniques with your choice of sampling distribution.

Generative Configuration (Temperature)

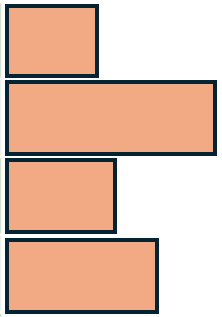
T = 1

apple	0.02
coffee	0.40
mojito	0.05
tea	0.10
……	……



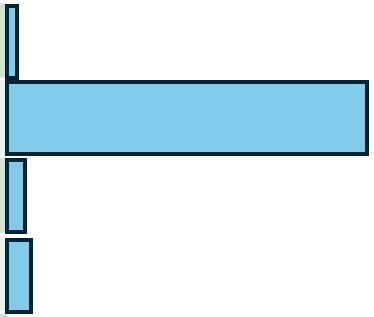
T >1 (higher temp)

apple	0.12
coffee	0.30
mojito	0.15
tea	0.20
……	……

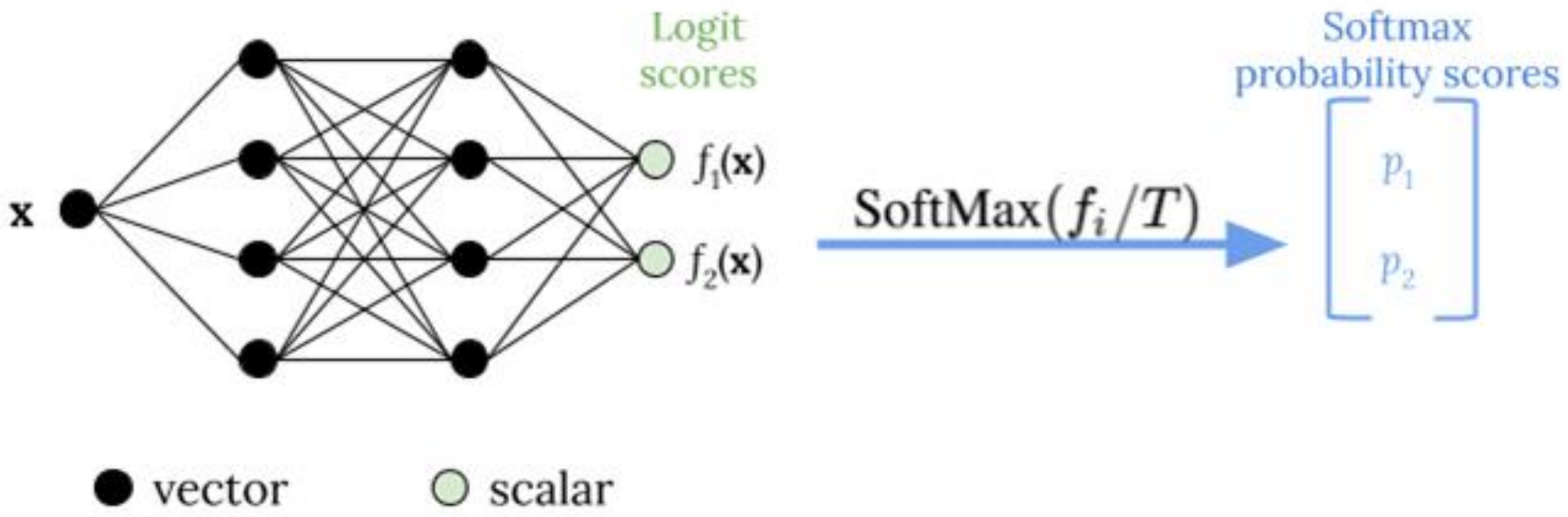


T < 1(cooler temp)

apple	0.015
coffee	0.900
mojito	0.011
tea	0.020
……	……



Generative Configuration (Temperature)



Courtesy: AWS documentation

Generative Configuration (Temperature)

- Temperature scales the logits, not the softmax probabilities.
- That's why services like ChatGPT, Grok caches the logits, not the softmax probs.
- Following code along with the notebook code can be used to verify that.

```
outputs = model(**inputs)
logits = outputs.logits[:, -1, :]
scaled_logits = logits / T
return torch.softmax(scaled_logits, dim=-1).squeeze()
```

Generative Configuration (Temperature)

- That’s why services like ChatGPT, Grok caches the logits, not the softmax probs.
- Note that Hugging Face Transformers doesn’t keep any attribute to directly get the softmax probabilities.

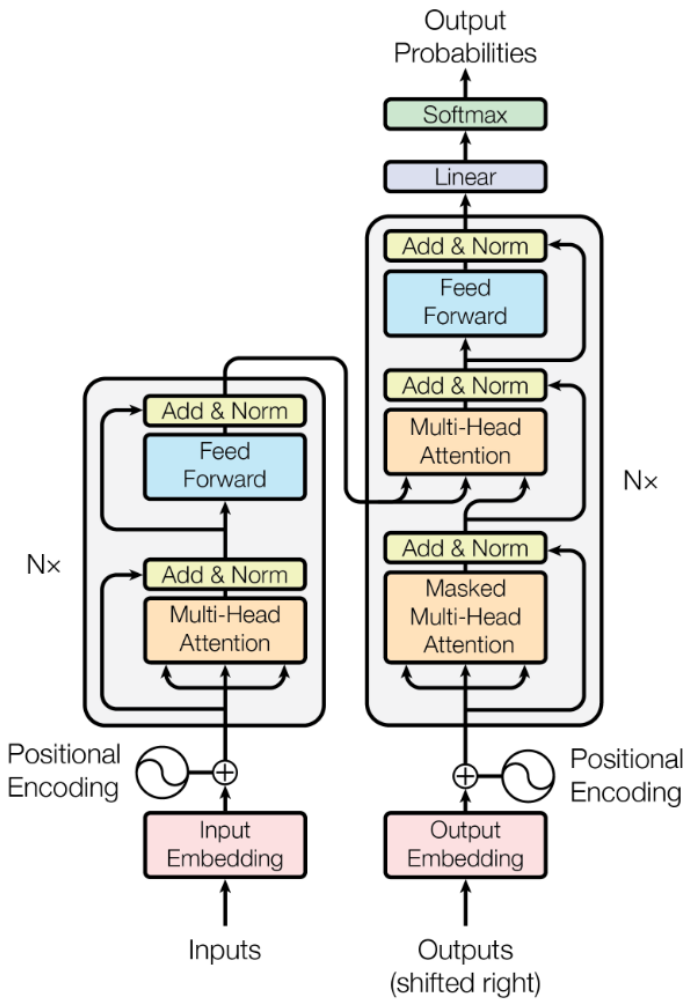
```
outputs = model(**inputs)
print(outputs.keys())

 >> dict_keys(['loss', 'logits', 'past_key_values’, 'hidden_states', 'attentions']
```

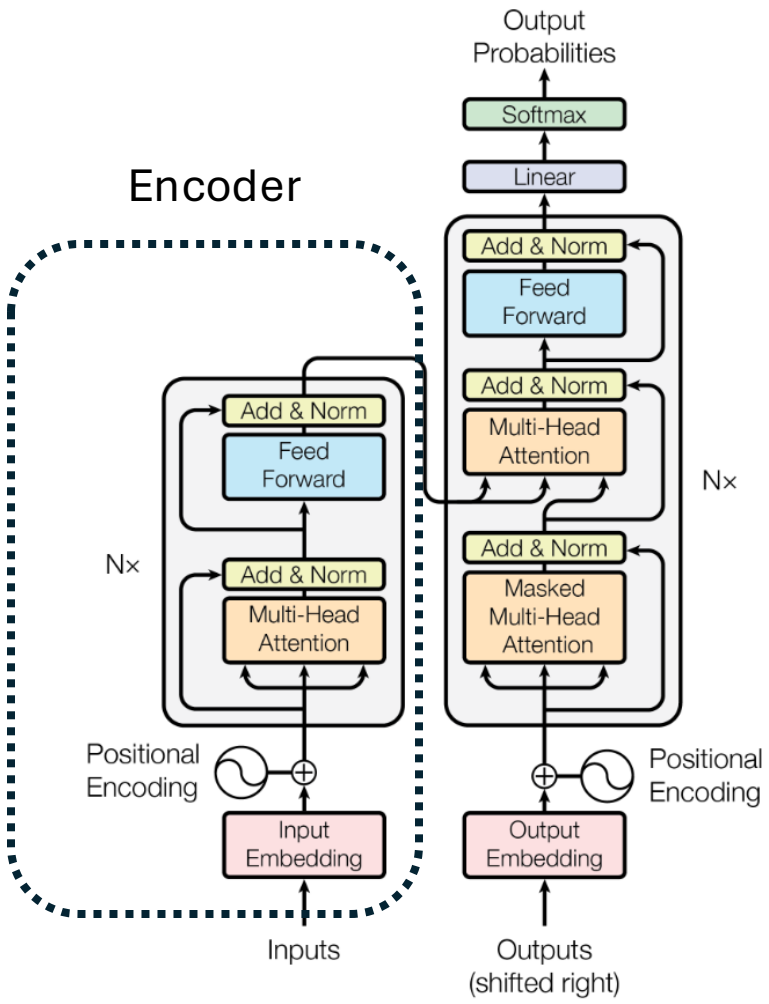
Generative Configuration (Flow in Practice)

- *Logit computation* *(API CALL)*
- *next_token_logits = next_token_logits / temperature* *(temperature)*
- *probs = torch.nn.functional.softmax(next_token_logits, dim=-1)* *(softmax)*
- *next_token = torch.multinomial(probs, num_samples=1)* *(top-p / top-K)*
- *max_new_tokens restricts the number of generated tokens* *(max_new_tokens)*

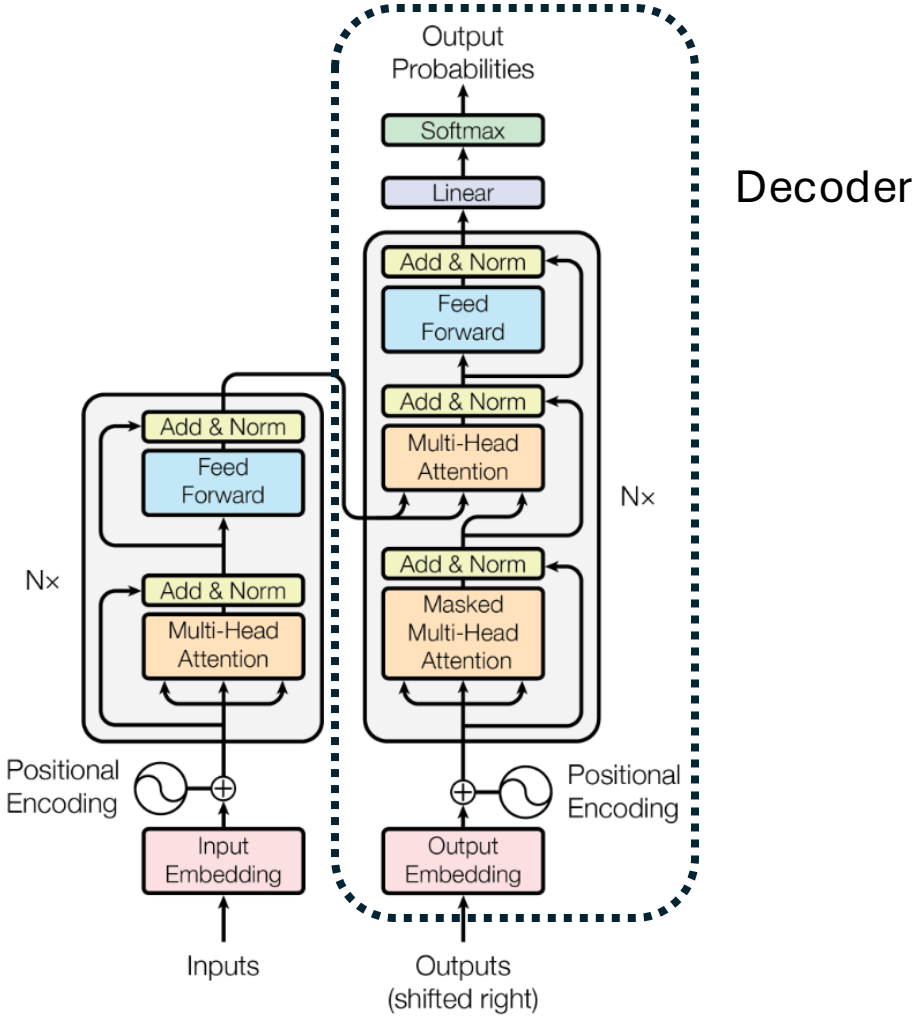
Transformer Architecture



Transformer Architecture



Transformer Architecture

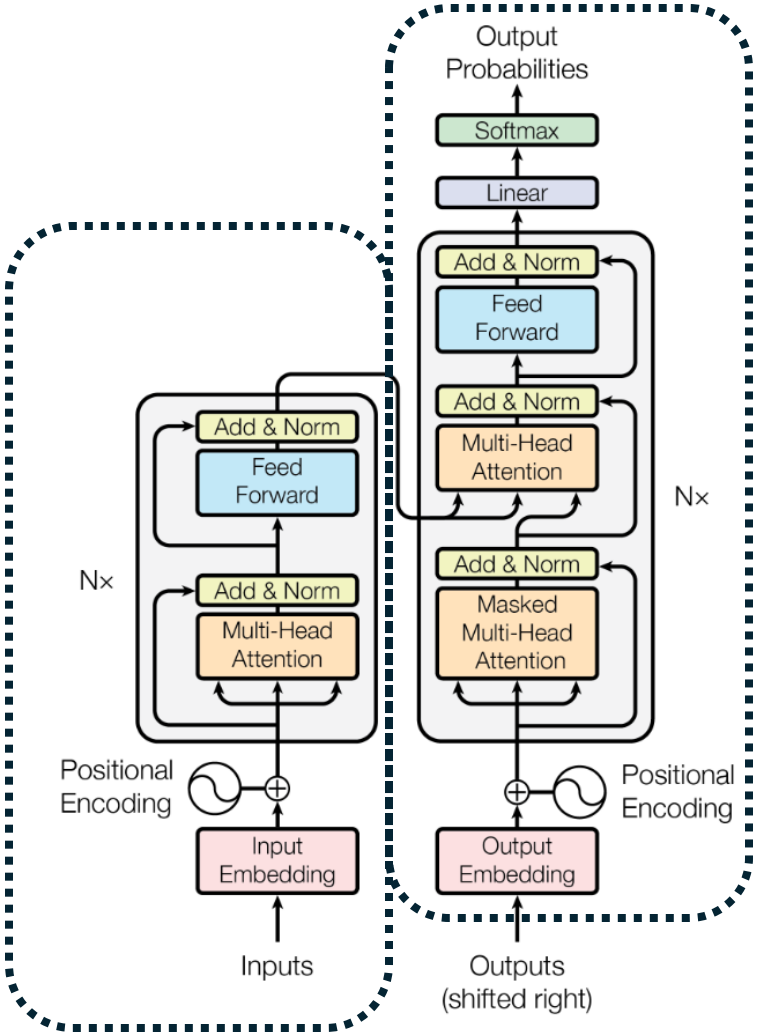


Decoder

Transformer Architecture

Encoder

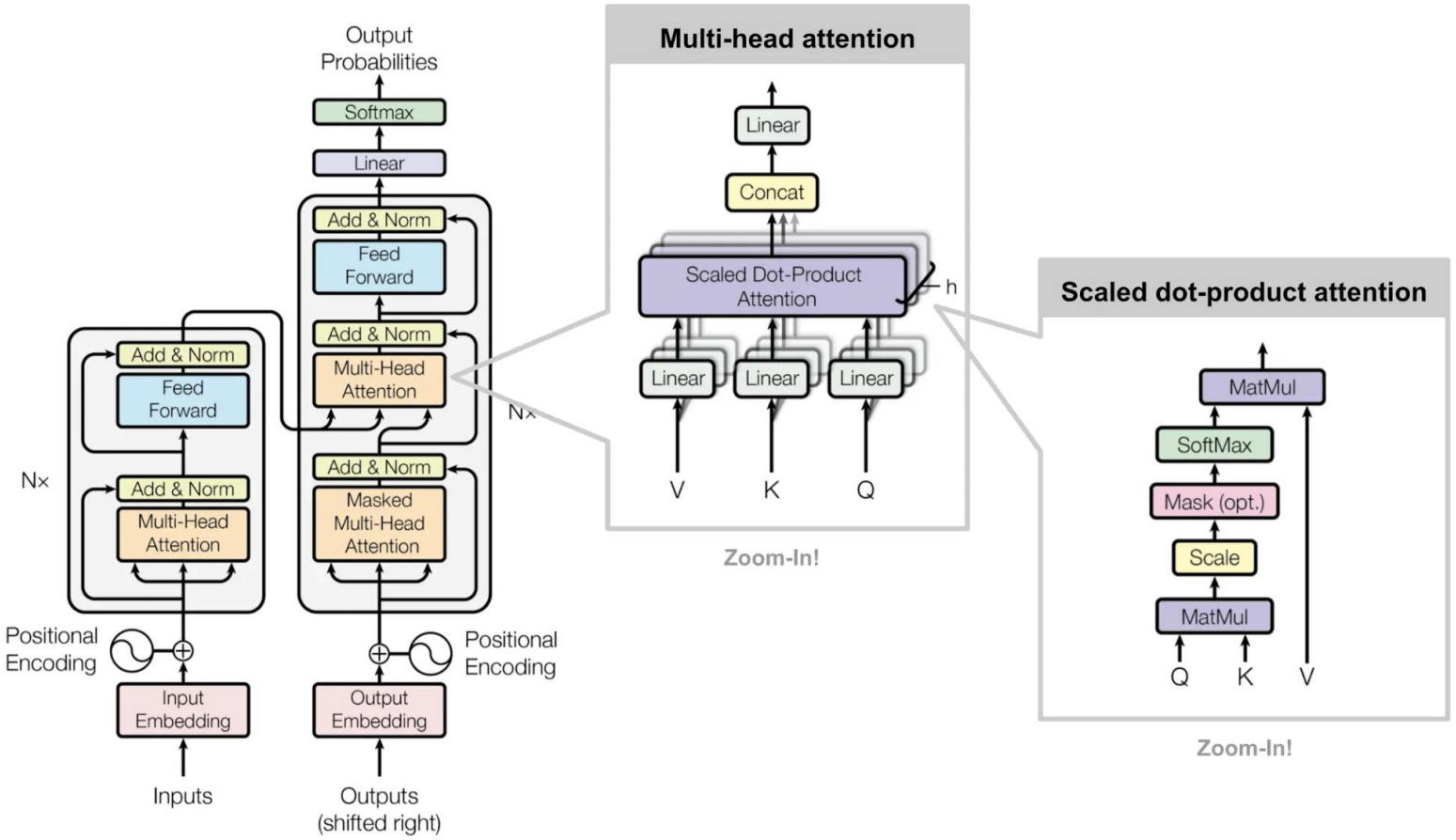
- Produces one vector per input
- Captures conceptualized representations



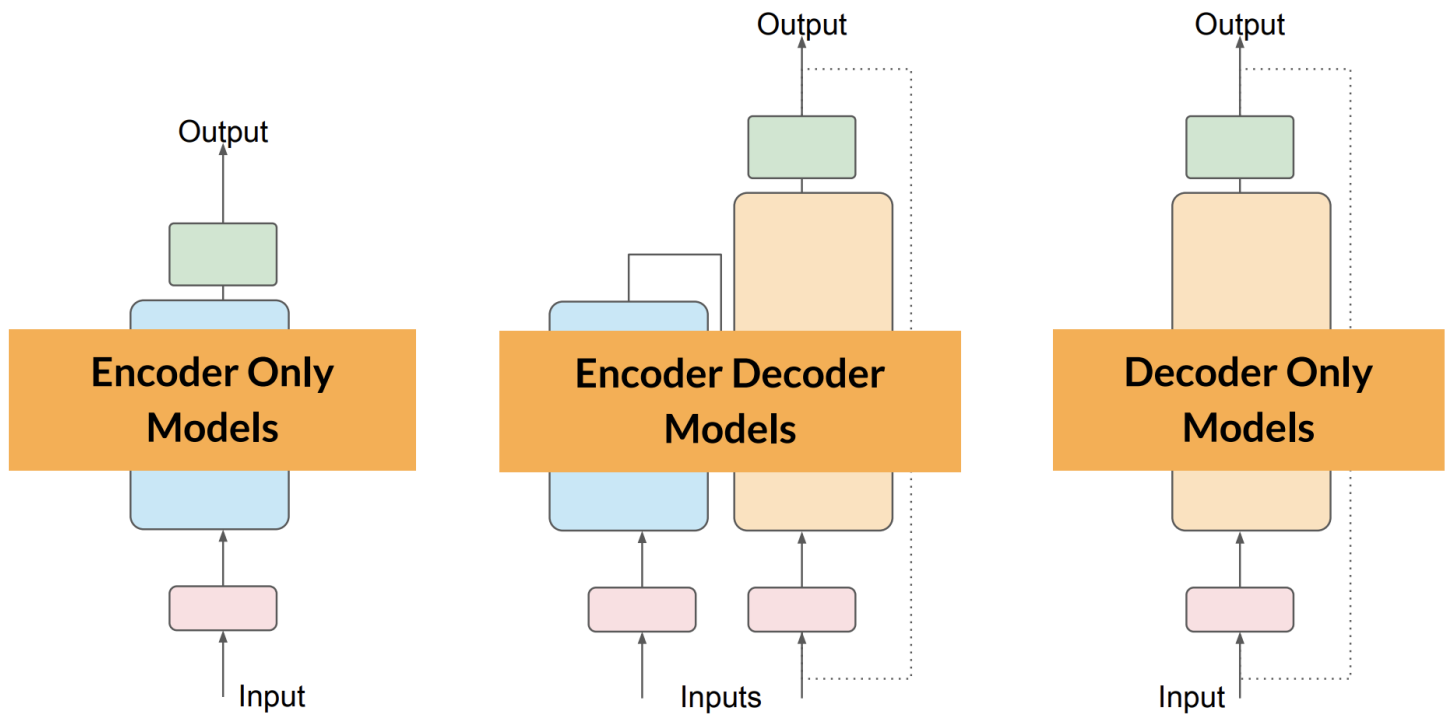
Decoder

- Accepts input tokens and generate new ones.
- Until <EOS> is generated.

Attention Mechanism



Transformers



Encoder-Only models

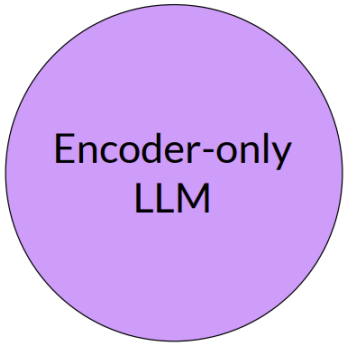
Original text

```
The teacher
teaches the
student.

[...]
```

Masked Language Modeling (MLM)

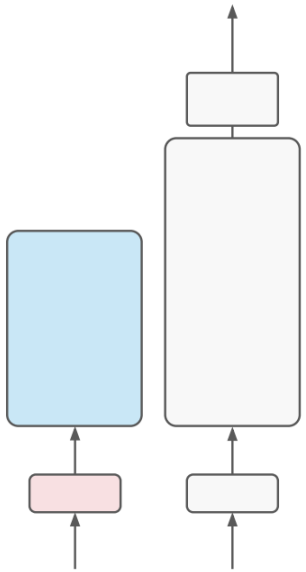
The	teacher	**<MASK>**	the	student



Objective: Reconstruct text ("denoising")

The	teacher	teaches	the	student

Bidirectional context



Encoder-Only models

- Training Method: Masked Language Modelling
- Examples: BERT, RoBERTa, ELECTRA
- Typical Use: seq-2-seq modelling, NER
- *Note:* Without any further changes in the architecture, the input and output size remains the same.

Assignment Teasers

- Code up top-p and top-K sampling techniques with your choice of sampling distribution.
- Add extra layers over small encoder-only model to convert it to a sentiment-analyser.

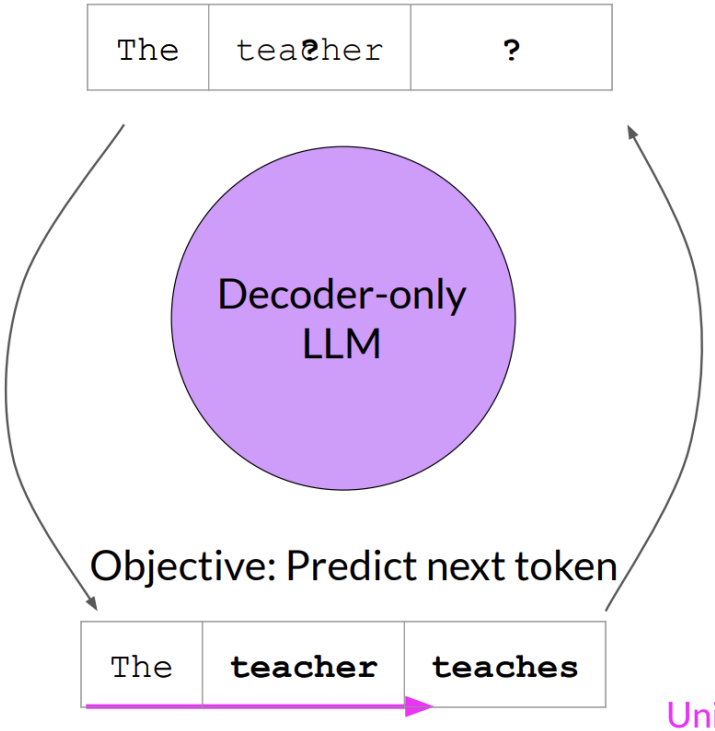
Decoder-Only models

Original text

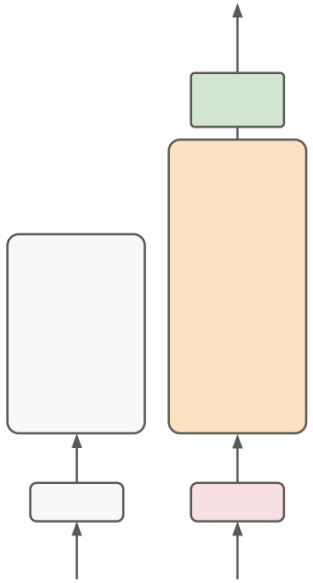
```
The teacher
teaches the
student.

[...]
```

Causal Language Modeling (CLM)



Unidirectional context



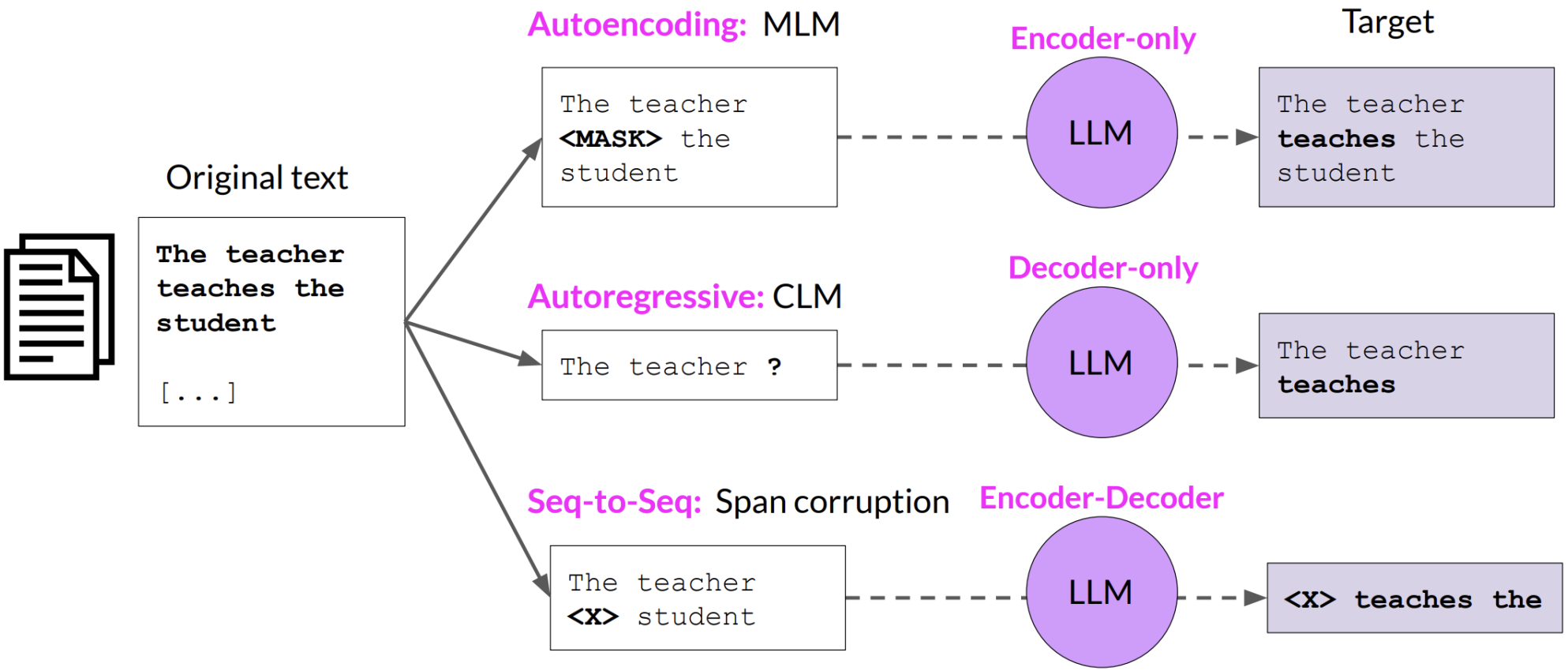
Decoder-Only models

- Training Method: Causal Language Modelling
- Examples: GPT-family, BLOOM, Llama-family
- Typical Use: Text generation for various downstream tasks.

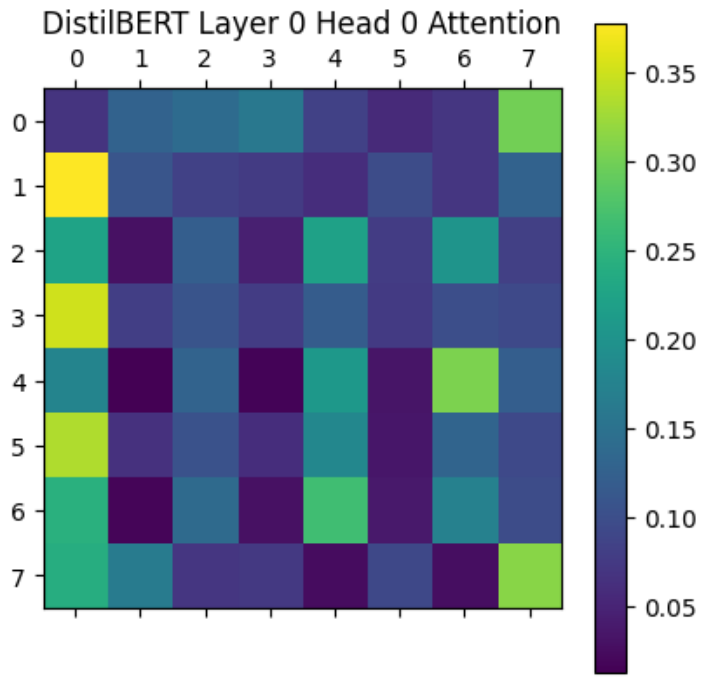
Encoder-Decoder models

- Training Method: Masked Sequence to Sequence Pre-training /Span correction
- Examples: BART, T5
- Typical Use: seq-2-seq modelling, translation, Summarization
- Input and Output Sequence lengths can be different.

Model Architectures and Training Objectives

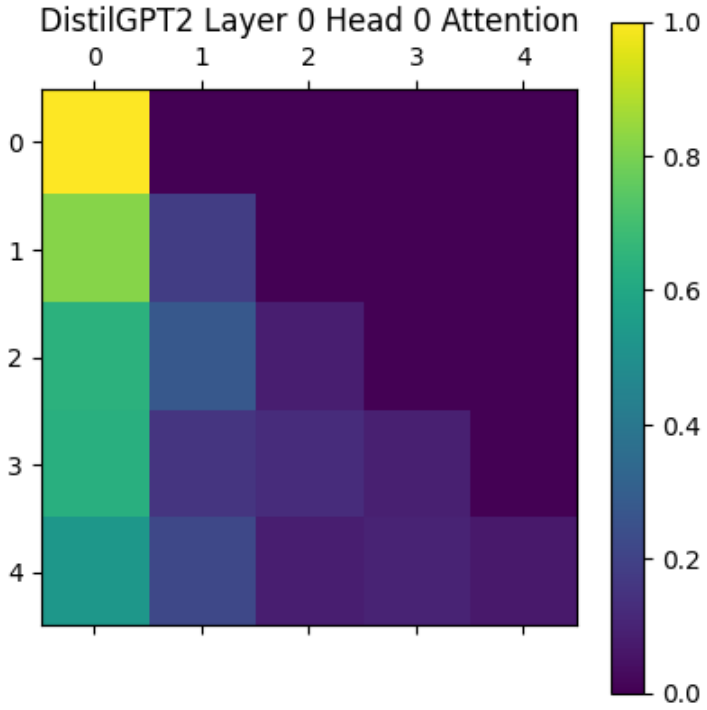


Attention Maps

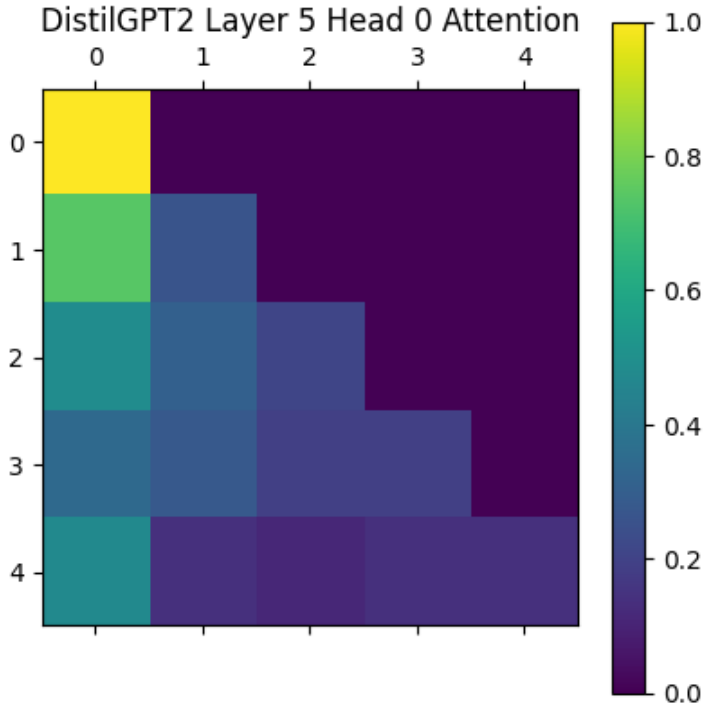


[CLS] the capital of France is Paris [SEP]

Attention Maps



Connections getting stronger and meaningful over layers



the capital of France is

Compute Required for Pretraining

	Bytes per parameter
Model Parameters (Weights)	4 bytes per parameter
Adam optimizer (2 states)	+8 bytes per parameter
Gradients	+4 bytes per parameter
Activations and temp memory (variable size)	+8 bytes per parameter (high-end estimate)
TOTAL	**=4 bytes per parameter** **+20 extra bytes per parameter**

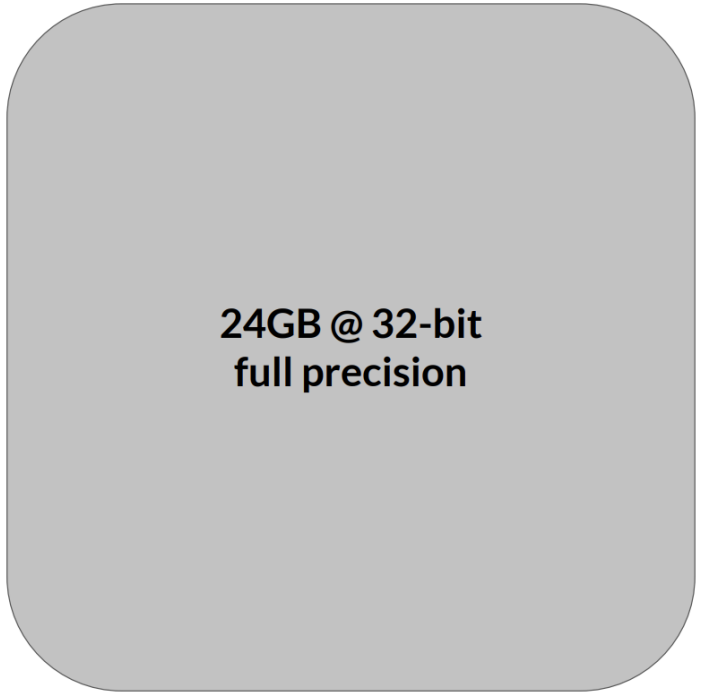
Sources: https://huggingface.co/docs/transformers/v4.20.1/en/perf_train_gpu_one#anatomy-of-models-memory, https://github.com/facebookresearch/bitsandbytes

Compute for Pretraining

Memory needed to store model

4GB @ 32-bit
full precision

Memory needed to train model



Compute for Pretraining

1B param model

175B param model

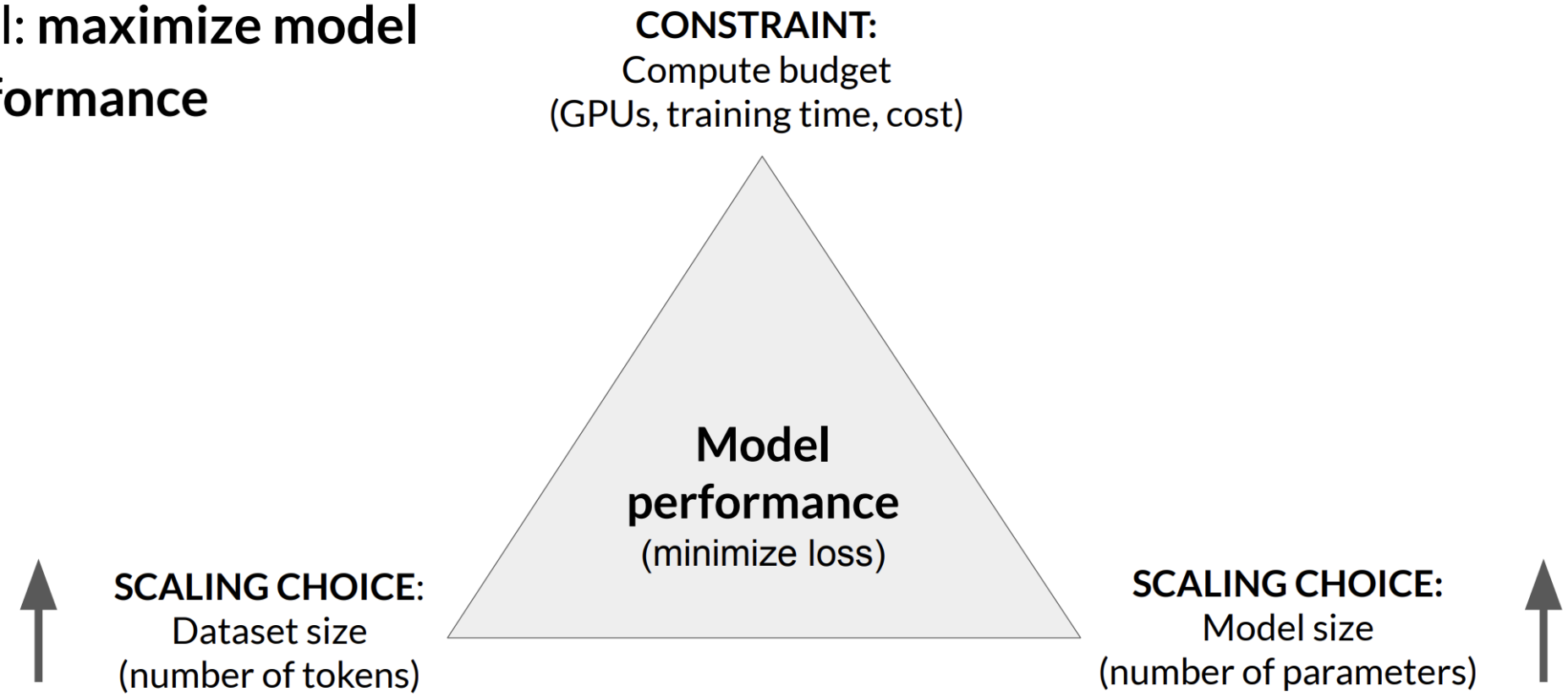
4,200 GB @ 32-bit full precision

500B param model

12,000 GB @ 32-bit full precision

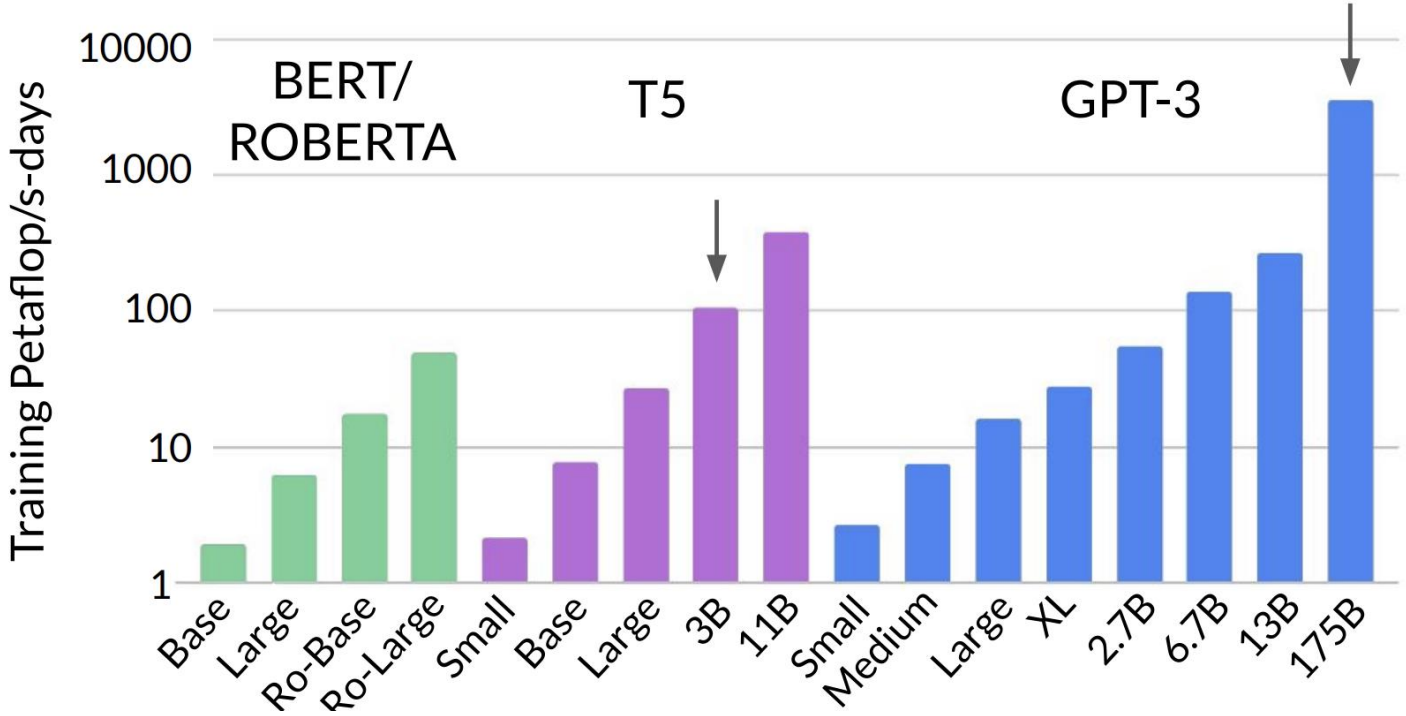
Scaling Choices for Pretraining

Goal: **maximize model performance**



Courtesy: Deeplearning.ai

Compute for Various LLMs



Courtesy: Brown et al., 2020 “Language models are few shot learners”

Scaling Law between Data and Model Size

Training Compute-Optimal Large Language Models

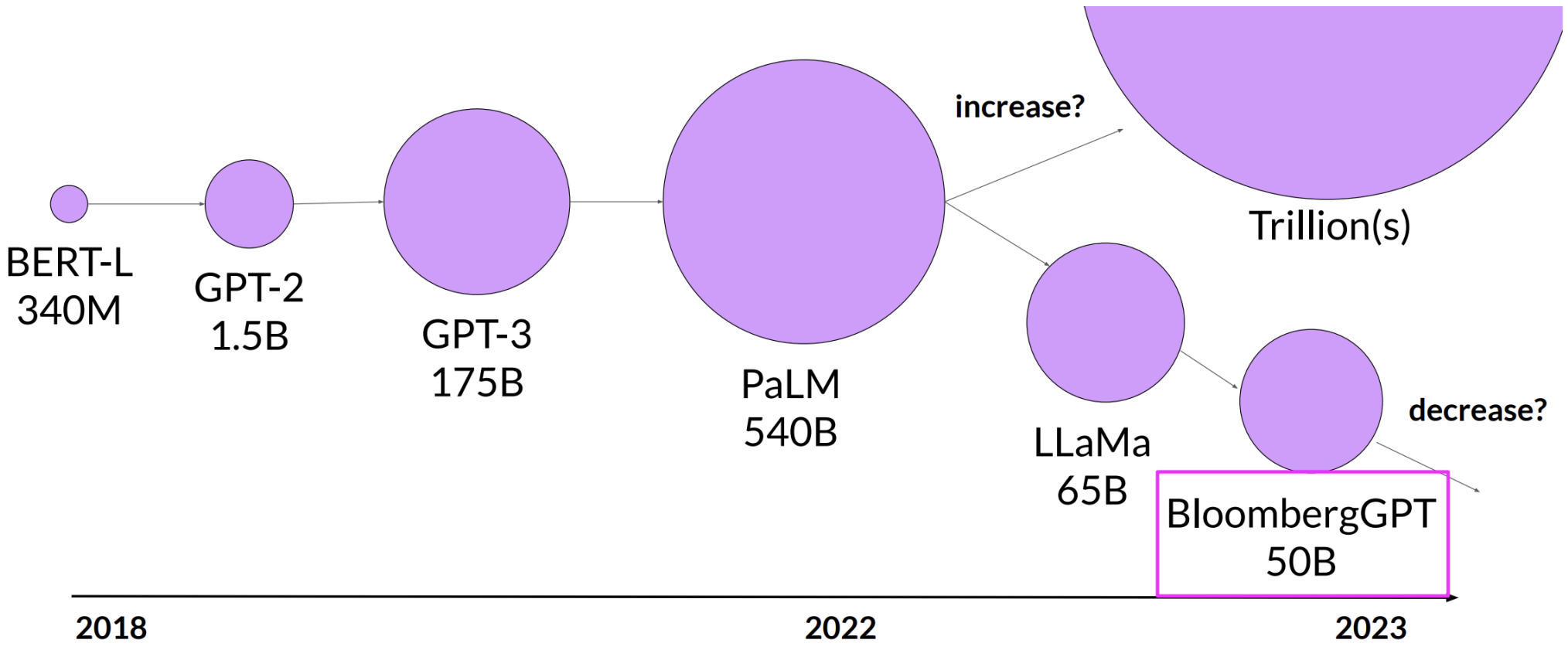
Jordan Hoffmann*, Sebastian Borgeaud*, Arthur Mensch*, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals and Laurent Sifre*

*Equal contributions

We investigate the optimal model size and number of tokens for training a transformer language model under a given compute budget. We find that current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the amount of training data constant. By training over 400 language models ranging from 70 million to over 16 billion parameters on 5 to 500 billion tokens, we find that for compute-optimal training, the model size and the number of training tokens should be scaled equally: for every doubling of model size the number of training tokens should also be doubled. We test this hypothesis by training a predicted compute-optimal model, *Chinchilla*, that uses the same compute budget as *Gopher* but with 70B parameters and 4× more more data. *Chinchilla* uniformly and significantly outperforms *Gopher* (280B), GPT-3 (175B), Jurassic-1 (178B), and Megatron-Turing NLG (530B) on a large range of downstream evaluation tasks. This also means that *Chinchilla* uses substantially less compute for fine-tuning and inference, greatly facilitating downstream usage. As a highlight, *Chinchilla* reaches a state-of-the-art average accuracy of 67.5% on the MMLU benchmark, greater than a 7% improvement over *Gopher*.

Courtesy: Jordan et al. 2022

Model Size inspired by Scaling Laws



Courtesy: Deeplearning.ai

Thank You